IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

YENEPOYA
(DEEMED TO BE UNIVERSITY)
Recognized under Sec 3(A) of the UGC Act 1956
Accredited by NAAC with 'A' Grade

Yenepoya (Deemed to be University)

Project – High Level Design

on

Identity Security Framework for Finance

Student Name:

Ayesha Tawfique

Industry Mentor:
Mr. Shashank Korram

Guided by:
Mr. Vikyath

Table of Contents

1. Introduction. 3
   1.1. Scope of the document. 3
   1.2. Intended Audience 3
   1.3. System overview. 3

2. System Design. 4
   2.1. Application Design 4
   2.2. Process Flow. 4
   2.3. Information Flow. 6
   2.4. Components Design 6
   2.5. Key Design Considerations 7
   2.6. API Catalogue. 7

3. Data Design. 9
   3.1. Data Model. 9
   3.2. Data Access Mechanism 10
   3.3. Data Retention Policies 11
   3.4. Data Migration 11

4. Interfaces 13

5. State and Session Management 16
   5.1. Blockchain State Management 16
   5.2. Decentralized Session Management 16
   5.3. Security and Revocation 17
   5.4. State Integrity Matrix 17

6. Caching 18

7. Non-Functional Requirements 19
   7.1. Security Aspects 19
   7.2. Performance Aspects 19

8. References 20

1. Introduction

The **Identity Security Framework for Finance** is a fully on-chain identity and access management solution implemented as a suite of four Solidity smart contracts targeting EVM-compatible blockchains. The system provides cryptographically verifiable user registration, Sign-In with Ethereum authentication, role-based access control, a two-factor authorization (2FA) mechanism for critical operations, and an immutable financial-grade audit trail, all without reliance on any off-chain backend.

1.1. Scope of the Document

This High-Level Design describes the architectural decisions, component structure, data model, interface catalogue, state management strategy, caching considerations, and non-functional requirements of the Identity Security Framework. It covers:

- The inheritance chain and responsibilities of all four contracts.
- Authentication and authorization process flows including Sign-In with Ethereum login and 2FA approval.
- On-chain data structures, enumerations, and event-based audit logging.
- Role-based permission boundaries and the permission matrix.
- Security, performance, and upgrade considerations.

1.2. Intended Audience

This document is intended for:

- Smart Contract Engineers and Auditors responsible for reviewing or extending the contract suite.
- Blockchain Architects making integration or upgrade decisions.
- Security Engineers assessing the threat model and security properties.
- Technical Project Managers and Product Owners requiring a system-level understanding.

1.3. System Overview

The system provides a permissioned, wallet-based identity registry on the blockchain. Users register their Ethereum wallet address against a KYC-derived identity hash. Authentication is performed via SIWE: the system generates a replay-protected signed message, and the user's ECDSA signature is verified on-chain using ecrecover. Critical administrative actions (role assignment, user deactivation) performed by non-owner Admins are gated by a time-limited (1-hour TTL) two-factor approval step requiring a secondary wallet or a fallback Admin approver.

The contract inheritance hierarchy:

IdentityStorage → AuditLogger → AccessController → IdentityManager

It enforces strict separation of concerns: storage is isolated at the base, audit logging is orthogonal, access control logic is centralized, and the public API surface is exposed only through the top-level IdentityManager.

2. System Design

2.1. Application Design

The system follows a layered monolithic contract architecture where each layer inherits from its predecessor, progressively adding capabilities while maintaining encapsulation. This pattern is preferred over multi-contract proxy architectures for auditability and simplicity, at the cost of upgradeability.

The four contracts form a strict linear inheritance chain:

- **IdentityStorage**: Pure storage layer. Defines all enumerations, structs, mappings, and primitive accessors. Contains no business logic or events.
- **AuditLogger**: Inherits IdentityStorage. Defines all nine blockchain events and the internal emit helpers. Acts as the audit backbone.
- **AccessController**: Inherits AuditLogger. Implements all access modifiers, SIWE message construction, ECDSA signature verification, and the 2FA approval lifecycle (create, approve, consume).
- **IdentityManager**: Inherits AccessController. Exposes the full public API. The only deployable contract; deployment via its constructor initializes the owner as an Admin.

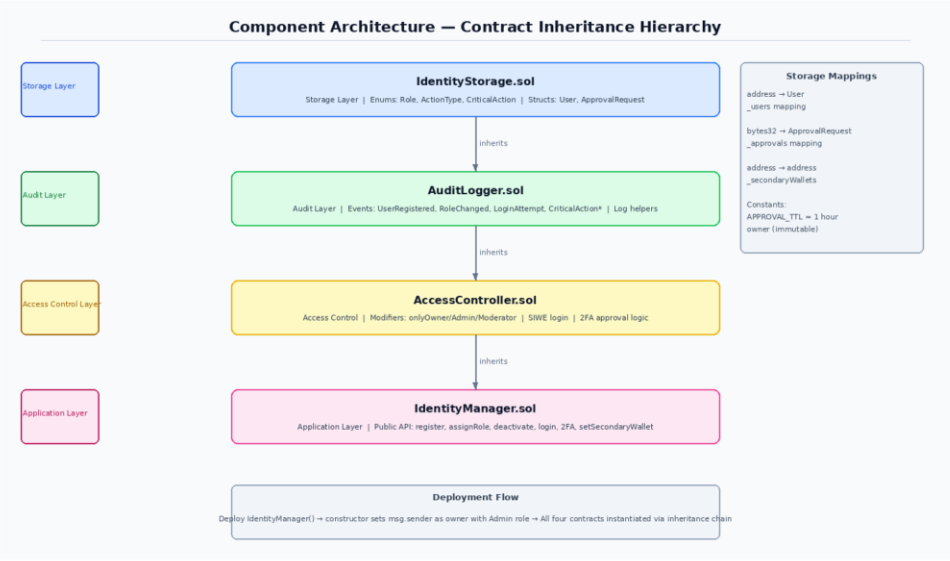

Figure 1 — Contract Component Architecture and Inheritance Hierarchy

2.2. Process Flow

The system supports two primary process flows: standard SIWE-based authentication, and the 2FA-gated critical action flow for sensitive administrative operations.

SIWE Authentication Flow

1. Client calls generateNonce(_user) to retrieve the SIWE challenge string containing wallet address, current nonce, contract address, and chain ID.
2. Client signs the message off-chain using their Ethereum wallet (MetaMask or equivalent) producing a 65-byte ECDSA signature.
3. Client calls verifySignature(signature); AccessController._verifyLogin() reconstructs the message, hashes it with the Ethereum prefix, and calls ecrecover to recover the signer.
4. On success: nonce is incremented (preventing replay), LoginAttempt(success=true) and ActionLogged events are emitted. On failure: LoginAttempt(success=false) is emitted and InvalidSignature is reverted.

2FA Critical Action Flow

1. Non-owner Admin calls requestCriticalAction(target, action, actionData); AccessController generates a deterministic approvalId from chain ID, contract address, requester, target, action, action data, and current nonce.
2. ApprovalRequest is stored on-chain with exists=true, isApproved=false, requestedAt=block.timestamp. CriticalActionRequested event is emitted.
3. Secondary wallet (or fallback Admin) calls approveCriticalAction(approvalId); AccessController validates approver identity, TTL not expired, and sets isApproved=true. CriticalActionApproved event emitted.
4. Admin calls the target operation (assignRole / deactivateUser); the requiresApproval modifier validates approval state and TTL, then deletes the approval record atomically after function execution.

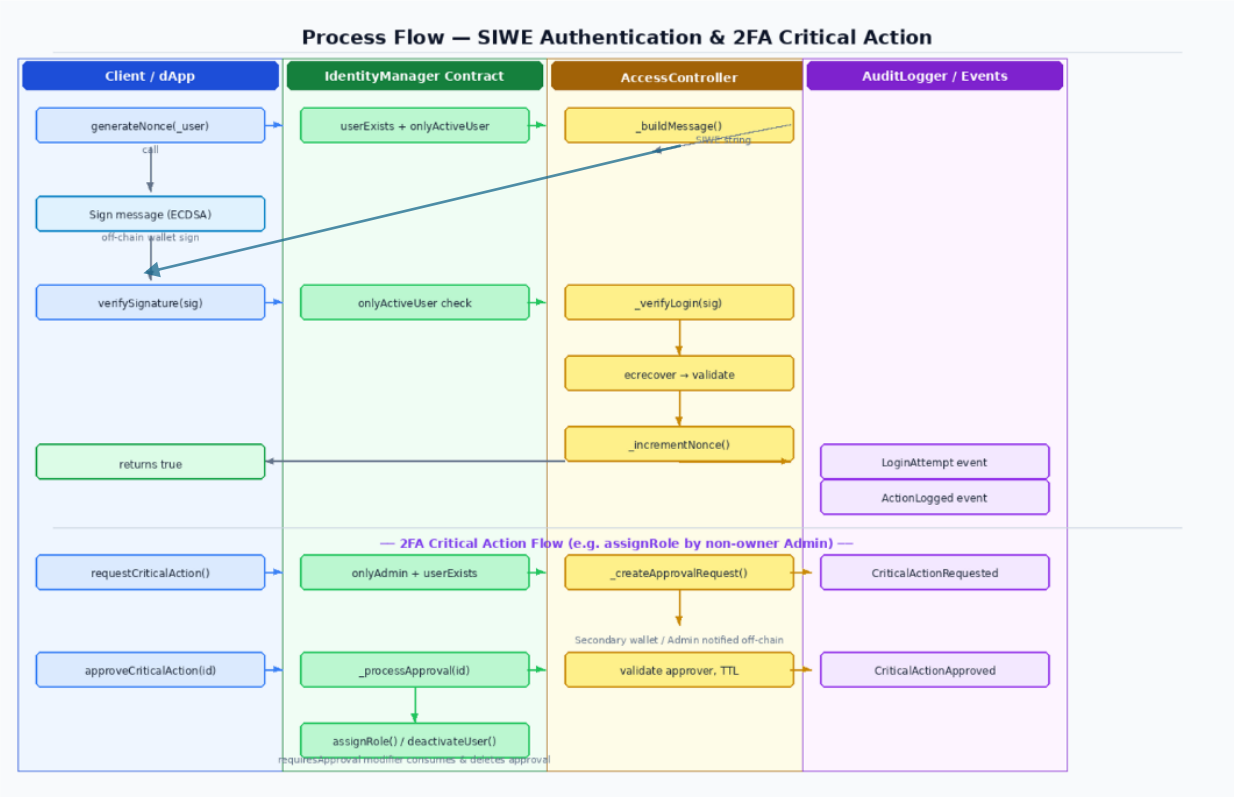

Figure 2 — SIWE Authentication and 2FA Critical Action Process Flow

2.3. Information Flow

All information in the system flows through on-chain transactions. There is no off-chain message bus or database. The information flow is:

- **Write path**: EOA wallet → EVM transaction → IdentityManager public function → AccessController modifier chain → IdentityStorage mapping writes → AuditLogger event emissions.
- **Read path**: Off-chain client / indexer → eth_call → IdentityManager view functions → IdentityStorage mapping reads → returned to caller.
- **Audit path**: All write operations emit indexed events. Clients and indexers subscribe to these events to build queryable off-chain indexes without modifying on-chain state.

2.4. Components Design

The four components and their key responsibilities are summarized below.

Contract	Layer	Key Responsibility	Access Modifier
IdentityStorage	Base	Structs, enums, mappings, primitive getters/setters	internal only
AuditLogger	Layer 2	Event definitions, emit helpers	internal only
AccessController	Layer 3	Modifiers, SIWE logic, ECDSA verification, 2FA lifecycle	internal only
IdentityManager	API	Public functions, constructor, deployment entry point	public/external

2.5. Key Design Considerations

- **Upgradeability Trade-off**: The monolithic inheritance pattern prioritizes auditability over upgradeability. A proxy pattern (UUPS or Transparent Proxy) was deliberately not used to eliminate proxy-related attack surfaces during this version. Upgrades require a new deployment and a migration mechanism.
- **Replay Attack Prevention**: Each user maintains an on-chain nonce. SIWE messages embed the current nonce; after successful authentication the nonce is incremented, making the consumed signature permanently invalid.
- **2FA Approval ID Determinism**: The approvalId is computed as keccak256(chainId, contract, requester, target, action, actionData, nonce), making it deterministic and pre-computable by clients without a round-trip, while remaining collision-resistant.
- **Owner Privilege Separation**: The contract owner (set at deploy time as an immutable address) bypasses 2FA requirements. This reduces operational friction for trusted deployers while allowing non-owner Admins to be constrained.
- **Nonce-as-Session**: Because nonces can be manually incremented (incrementNonce()), users can invalidate all previously issued SIWE challenges on demand, providing a session revocation mechanism.

2.6. API Catalogue

All public functions are exposed by IdentityManager. Inherited abstract functions are not directly callable.

Function	Access	Parameters	Returns	Description
registerUser()	Public	bytes32 identityHash	bool	Self-registration with KYC identity hash. Reverts if already registered or hash is zero.
generateNonce()	Public view	address _user	string	Returns SIWE challenge string for the given active, registered user.
verifySignature()	External	bytes signature	bool	Verifies 65-byte ECDSA signature, increments nonce, emits login events.
assignRole()	onlyAdmin	address _user, Role _newRole	bool	Assigns role. Owner: direct. Non-owner Admin: requires prior 2FA approval.
deactivateUser()	onlyMod+	address _user	bool	Deactivates user. Owner: direct. Others: requires prior 2FA approval.
reactivateUser()	onlyAdmin	address _user	bool	Reactivates an inactive user. No 2FA required.
incrementNonce()	Registered	—	uint256	Manually increments caller nonce; invalidates all pending SIWE challenges.
requestCriticalAction()	onlyAdmin	address target, CriticalAction	bytes32	Creates 2FA approval request; returns approvalId.

		action, bytes32 data		
approveCriticalAction()	Secondary/ Admin	bytes32 approvalId	—	Marks a pending approval as approved. Validates approver and TTL.
setSecondaryWallet()	Registered	address secondary	—	Sets 2FA secondary approver wallet for the caller.
getSecondaryWallet()	View	address _user	address	Returns secondary wallet address for a user.
getApprovalRequest()	View	bytes32 approvalId	ApprovalRequest	Returns full approval request struct.
buildApprovalId()	View	address req, address tgt, CriticalAction, bytes32 data	bytes32	Pre-computes approval ID deterministically.
getUserDetails()	View	address _user	User	Returns full User struct for a registered user.
isRegistered()	View	address _user	bool	Returns true if wallet is registered.
getRole()	View	address _user	Role	Returns current role enum value.
hasRole()	View	address _user, Role	bool	Returns true if user has the specified role and is active.
getNonce()	View	address _user	uint256	Returns current nonce value.
isActiveUser()	View	address _user	bool	Returns true if user is registered and active.

3. Data Design

3.1. Data Model

All data is stored on-chain in EVM contract storage slots. There is no relational database. The data model consists of two primary structs, three mappings, three enumerations, and two constants.

User Struct

Field	Type	Description
walletAddress	address	Primary key — the user's Ethereum wallet address. Zero address indicates unregistered.
role	Role	Current role: None (0), User (1), Moderator (2), Admin (3).
identityHash	bytes32	Off-chain KYC identity commitment hash. Owner's hash is bytes32(0).
isActive	bool	True if the user is active and permitted to operate.
nonce	uint256	Monotonically increasing counter for SIWE replay protection.

ApprovalRequest Struct

Field	Type	Description
requester	address	Admin wallet that initiated the critical action request.
target	address	Target user wallet the action will be applied to.
action	CriticalAction	Enum: RoleChange (0) or Deactivation (1).
actionData	bytes32	ABI-encoded action payload (e.g., encoded new role value or target address).
isApproved	bool	Set to true by the secondary wallet / fallback Admin upon approval.
exists	bool	Sentinel flag. Distinguishes zero-value struct from an actual request.
requestedAt	uint256	block.timestamp at creation; used with APPROVAL_TTL for expiry checks.

Storage Mappings

Mapping	Key Type	Value Type	Notes
_users	address	User	Private; accessed only via internal getters/setters in IdentityStorage.
_approvals	bytes32	ApprovalRequest	Internal; keyed by deterministic approvalId hash.
_secondaryWallets	address	address	Internal; maps primary wallet to its 2FA secondary wallet.

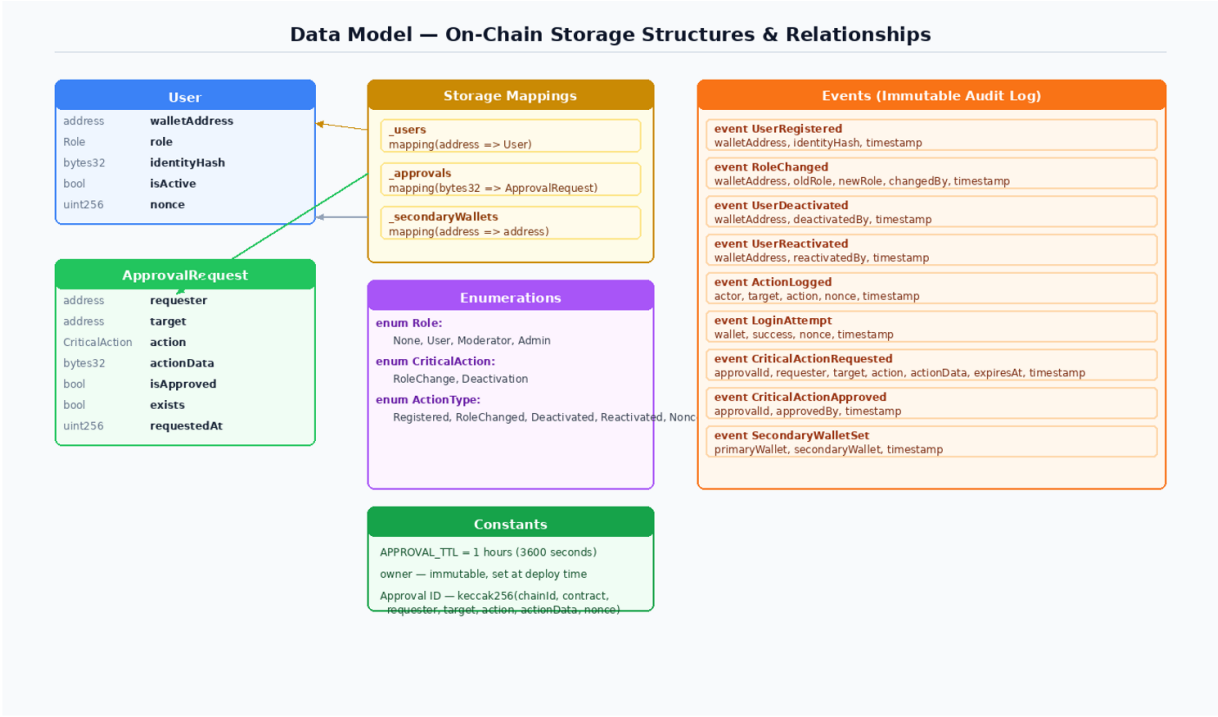

Figure 4 — On-Chain Data Model, Structs, Mappings, Enumerations, and Events

3.2. Data Access Mechanism

IdentityStorage exposes a controlled set of internal getter and setter functions. No mapping is accessed directly outside of this contract. This ensures that all read and write operations pass through a single, auditable code path.

Accessor	Type	Description
_getUser(address)	view	Returns User memory copy (snapshot at call time).
_getUserStorage(address)	view	Returns User storage reference for in-place modification.
_setUser(address, User)	write	Full user record write; used only during registration.
_exists(address)	view	Returns walletAddress != address(0); cheap existence check.
_isActive(address)	view	Returns isActive field.
_getRole(address)	view	Returns role enum.
_getNonce(address)	view	Returns current nonce.
_setRole(address, Role)	write	Targeted role field update.
_setActive(address, bool)	write	Targeted isActive field update.
_incrementNonce(address)	write	Uses unchecked arithmetic for gas efficiency; overflow is not a concern at realistic nonce values.
_storeApproval / _getApproval / _markApproved / _deleteApproval	mixed	Full CRUD for ApprovalRequest entries keyed by approvalId.
_getSecondaryWallet / _setSecondaryWallet	mixed	Secondary wallet address read/write.

3.3. Data Retention Policies

Blockchain storage is permanent and immutable. The following retention characteristics apply:

- **User records**: Persist indefinitely once registered. Deactivation sets isActive=false; the record is never deleted. Wallet addresses and identity hashes are permanently on-chain.
- **Approval requests**: Single-use with automatic deletion. The requiresApproval modifier calls _deleteApproval() after the guarded function executes, releasing the storage slot (gas refund) and preventing approval replay.

- **Event logs**: Emitted events are stored in transaction receipts and are permanent, but are not accessible from within the EVM. They serve as the immutable, append-only audit trail.
- **Secondary wallets**: Persist until overwritten by a new setSecondaryWallet() call. There is no explicit deletion; setting to address(0) would clear the mapping (the contract does not currently restrict this).

3.4. Data Migration

The system does not implement an upgradeable proxy pattern, and therefore has no built-in data migration mechanism. In the event of a required upgrade:

- A new IdentityManager contract would be deployed.
- Off-chain migration scripts would replay registration and role events from the old contract's event log (UserRegistered, RoleChanged) to reconstruct the user registry in the new contract.
- A governance mechanism (multi-sig or DAO vote) would be required to designate the new contract as authoritative and deprecate the old one.
- Identity hashes are already stored on-chain and can be replayed without re-submission of KYC data.

Future versions can implement an ERC-1967 UUPS proxy with a separate storage contract to support in-place upgrades without data migration.

4. Interfaces

The system exposes interfaces at three distinct levels: the on-chain ABI, emitted events (as a data interface for off-chain consumers), and custom error types for precise revert decoding.

On-Chain ABI Interface

All externally callable functions are declared in IdentityManager. There is no separate interface (IIdentityManager) file in the current implementation, though one is recommended for future integration consumers.

Event Interface

Events form the primary data interface for off-chain systems (indexers, dashboards, compliance tools). All events are defined in AuditLogger and emitted through internal helpers to ensure consistent indexing.

Event	Indexed Parameters	Non-Indexed Parameters	Trigger
UserRegistered	walletAddress	identityHash, timestamp	registerUser() success
RoleChanged	walletAddress, oldRole, newRole	changedBy, timestamp	assignRole() success
UserDeactivated	walletAddress	deactivatedBy, timestamp	deactivateUser() success
UserReactivated	walletAddress	reactivatedBy, timestamp	reactivateUser() success
ActionLogged	actor, target, action	nonce, timestamp	All state-changing operations
LoginAttempt	wallet	success, nonce, timestamp	verifySignature(); both success and failure

CriticalActionRequested	approvalId, requester, target	action, actionData, expiresAt, timestamp	requestCriticalAction()
CriticalActionApproved	approvalId	approvedBy, timestamp	approveCriticalAction()
SecondaryWalletSet	primaryWallet, secondaryWallet	timestamp	setSecondaryWallet()

Custom Error Interface

The system uses Solidity custom errors instead of revert strings, reducing gas costs and enabling precise error decoding by clients.

Error	Parameters	Condition
NotOwner	—	Caller is not the contract owner
NotAdmin	—	Caller lacks Admin role or is inactive
NotModerator	—	Caller is below Moderator level
NotRegistered	address user	Target wallet is not registered
AlreadyRegistered	address user	registerUser() called for an existing wallet
UserNotActive	address user	Operation requires an active user
UserAlreadyActive	address user	reactivateUser() called on an already-active user
ZeroAddress	—	address(0) passed where not permitted
EmptyIdentityHash	—	bytes32(0) passed as identityHash
CannotAssignNoneRole	—	Role.None is not a valid assignment target
SameRoleAssigned	—	Target already holds the requested role
CannotSelfDeactivate	—	User attempts to deactivate themselves
CannotDeactivateOwner	—	Attempt to deactivate the contract owner
CannotDowngradeOwner	—	Attempt to change owner's role away from Admin

UnauthorizedRoleChange	—	Non-owner attempts to assign Admin role
InvalidSignature	—	ecrecover returns address(0) or mismatched signer
InvalidSignatureLength	—	Signature is not exactly 65 bytes
ApprovalNotFound	bytes32 approvalId	No approval request exists for the given ID
AlreadyApproved	bytes32 approvalId	approveCriticalAction() called on an already-approved request
NotAuthorizedApprover	bytes32, address	Caller is not the valid secondary wallet or fallback Admin
ApprovalExpired	bytes32 approvalId	block.timestamp > requestedAt + APPROVAL_TTL (1 hour)
ApprovalRequired	bytes32 approvalId	Operation requires an approved 2FA request
InvalidSecondaryWallet	—	Secondary wallet is caller's own address or the owner address

5. State and Session Management

5.1. Blockchain State Management

- State in the system is managed entirely through EVM contract storage (SSTORE/SLOAD). There are no off-chain sessions or caches that are authoritative. The canonical state at any point is the current contents of the three storage mappings (_users, _approvals, _secondaryWallets) plus the immutable owner address.
- State transitions are atomic within a single transaction. EVM's all-or-nothing execution semantics guarantee that a partial state update (e.g., a role change without the corresponding event emission) cannot occur. All modifiers execute before the function body, ensuring guard conditions are checked before any state is written.

5.2. Decentralized Session Management

The system does not maintain traditional HTTP sessions. Instead, SIWE-based authentication is stateless at the smart contract level: each login is an independent, signed, nonce-validated transaction. The nonce plays the role of a session token:

- A valid unused nonce represents an active, claimable authentication token.
- Consuming the nonce (via verifySignature) invalidates the session, analogous to a one-time token.
- Calling incrementNonce() invalidates any outstanding SIWE challenge without requiring a full authentication round-trip.

Off-chain session persistence (JWT, cookie, API token) is the responsibility of the client application layer and is out of scope for this contract suite. The contract provides the cryptographic ground truth that off-chain sessions should anchor to.

5.3. Security and Revocation

Several on-chain revocation mechanisms are provided:

- User deactivation (isActive = false) immediately revokes all effective permissions. All modifiers check _isActive() before proceeding.
- Nonce increment invalidates any pending SIWE challenge. Clients should call incrementNonce() when a session compromise is suspected.
- Approval TTL (1 hour) automatically expires unexecuted 2FA approvals, preventing stale approvals from being used after the intended action window.
- Secondary wallet removal; updating secondaryWallet to a different address immediately revokes the previous secondary approver's ability to approve future critical actions (in-flight approvals are unaffected as the approvalId is already stored).

5.4. State Integrity Matrix

State Transition	Guard Conditions	State Written	Events Emitted
User Registration	!exists, identityHash!=0	_users[addr] = User{...}	UserRegistered, ActionLogged(Registered)
SIWE Login Success	exists, isActive, signature valid	nonce++	LoginAttempt(true), ActionLogged(LoginSuccess)
SIWE Login Failure	exists, isActive, signature invalid	(none)	LoginAttempt(false), ActionLogged(LoginFailed)
Role Assignment (Owner)	onlyOwner, exists, valid role	_users[addr].role = newRole	RoleChanged, ActionLogged(RoleChanged)
Role Assignment (Admin)	onlyAdmin, 2FA approved, valid role	_users[addr].role, del _approvals[id]	RoleChanged, ActionLogged(RoleChanged)

User Deactivation	onlyMod+, 2FA approved (non-owner)	_users[addr].isActive = false	UserDeactivated, ActionLogged(Deactivated)
User Reactivation	onlyAdmin, exists, inactive	_users[addr].isActive = true	UserReactivated, ActionLogged(Reactivated)
2FA Request Created	onlyAdmin, !owner, exists target	_approvals[id] = ApprovalRequest{...}	CriticalActionRequested, ActionLogged(CriticalRequested)
2FA Approval Given	valid approver, !expired, !already approved	_approvals[id].isApproved = true	CriticalActionApproved, ActionLogged(CriticalApproved)

6. Caching

Blockchain contracts do not natively support traditional caching layers. However, several caching strategies are relevant at different levels of the stack.

In-Memory EVM Caching

Solidity distinguishes between storage references and memory copies. The _getUser() function returns a User memory copy, which is cheaper for read-only consumption but does not reflect in-flight state changes within the same transaction. The _getUserStorage() accessor returns a storage reference for in-place mutation, avoiding the gas cost of a full struct re-write.

Off-Chain Indexer Caching

For read-heavy use cases (e.g., dashboards displaying all registered users and their roles), querying the contract storage slot-by-slot is prohibitively expensive. The mentioned approach is to index the nine emitted events using a custom EVM log indexer. The indexer maintains a materalized view of current state by processing events sequentially. All state-changing functions emit ActionLogged events that can reconstruct the full audit timeline.

Event-driven caching guarantees: UserRegistered provides the ground truth for existence. Subsequent RoleChanged, UserDeactivated, and UserReactivated events maintain current state. ActionLogged with ActionType.LoginSuccess / LoginFailed provides a full session audit log. CriticalAction events provide a 2FA approval audit trail.

Client-Side Nonce Caching

Clients may cache the current nonce value (returned by getNonce()) to construct SIWE messages without an additional on-chain call. However, nonce staleness is possible if another party increments the nonce concurrently. Clients should always re-fetch the nonce on authentication failure to handle this race condition.

Approval ID Pre-Computation

The buildApprovalId() view function allows clients to compute the expected approvalId deterministically before submitting requestCriticalAction(). This pre-computation can be cached client-side and used to monitor for the corresponding CriticalActionRequested event, enabling reactive UX without polling.

7. Non-Functional Requirements

7.1. Security Aspects

Security Property	Implementation	Notes
Replay Attack Prevention	Per-user monotonic nonce embedded in SIWE message	Nonce incremented atomically on successful authentication
Signature Validation	On-chain ecrecover with EIP-191 prefix	65-byte length enforced; v-value normalization applied
2FA Authorization	Time-limited (1hr TTL) approval request keyed by deterministic hash	Approval deleted after use; cannot be replayed
Role-Based Access	Modifiers enforced at EVM level before function execution	Cannot be bypassed without contract modification
Owner Privilege Segregation	Owner immutable at deploy time; bypasses 2FA	No function allows owner demotion except by owner (CannotDowngradeOwner guard)
Self-Action Prevention	CannotSelfDeactivate check; InvalidSecondaryWallet prevents self-as-secondary	Prevents privilege manipulation via self-approval
Integer Overflow	Solidity 0.8.x built-in overflow checks; nonce uses unchecked (justified)	Unchecked nonce increment is safe: uint256 overflow at 2^256 iterations is practically impossible
Denial of Service	No loops over unbounded arrays; all operations are O(1) storage lookups	Registry design avoids iteration-based DoS vectors
Chain Replay	approvalId includes block.chainid	Prevents replay of 2FA approvals across EVM forks

7.2. Performance Aspects

Operation	Gas Complexity	Optimization Notes
registerUser()	~50,000 gas	Single SSTORE for new User struct (5 fields)
verifySignature()	~30,000–40,000 gas	ecrecover is a precompile (~3,000 gas); nonce increment + 2 events dominate
generateNonce()	~3,000 gas	Pure view; SLOAD only
requestCriticalAction()	~60,000 gas	New ApprovalRequest SSTORE (7 fields) + event
approveCriticalAction()	~25,000 gas	Single SSTORE (isApproved flag) + event
assignRole() / deactivateUser()	~30,000–40,000 gas (with prior 2FA)	SSTORE (role/active flag) + SDELETE (approval) + event
View functions (get*)	~3,000–5,000 gas	SLOAD operations; free when called via eth_call
incrementNonce()	~15,000 gas	Single SSTORE + event; unchecked arithmetic saves ~200 gas

All write operations are O(1) in gas complexity, there are no loops, no unbounded arrays, and no iteration over the user registry. The system is designed to scale to any number of registered users without per-user gas growth affecting unrelated operations.

8. References

- Ethereum Foundation. (2021). **EIP-4361: Sign-In with Ethereum**. SIWE Standard. https://eips.ethereum.org/EIPS/eip-4361
- Ethereum Foundation. (2016). **EIP-191: Signed Data Standard**. Ethereum Prefixed Message Hashing, \x19Ethereum Signed Message. https://eips.ethereum.org/EIPS/eip-191
- Solidity Documentation. (2023). **Solidity 0.8.19 Documentation**. Language Reference. https://docs.soliditylang.org
- OpenZeppelin. (2023). **Access Control Patterns**. Role-based Access Reference. https://docs.openzeppelin.com/contracts
- Vogelsteller, F. (2017). **ERC-725: General Data Key/Value Store**. Identity Standard Reference for On-chain Identity Anchoring. https://eips.ethereum.org/EIPS/eip-725
- Wood, G. (2024). **Ethereum: A Secure Decentralised Generalised Transaction Ledger (Yellow Paper)**. Ethereum Virtual Machine Specification, Gas Costs Reference. https://ethereum.github.io/yellowpaper/paper.pdf
- Smart Contract Weakness Classification. (2023). **SWC Registry**. Security Audit Reference. https://swcregistry.io